QuizForge

Application Development Report

Java · Operating Systems · Dev Environment

Document Type	Technical Application Report
Version	2.0 — With Authentication System
Date	May 2026
Status	Production-Ready
Scope	Full-stack web application with role-based login
Technology	HTML5 · CSS3 · Vanilla JavaScript

1. Executive Summary

QuizForge is a self-contained, browser-based quiz application designed for academic assessment in Computer Science courses. The platform delivers randomised multiple-choice quizzes spanning three technical domains: Java Programming, Operating Systems, and Development Environment & Tooling.

Version 2.0 introduces a complete role-based authentication system, enabling institutions to manage student accounts, track performance over time, and provide administrators with a centralised oversight dashboard — all without requiring a backend server or database.

150 Questions in Bank	**3** Topic Domains	**2** User Roles
50 Questions per Topic	**5** Quiz Size Options	**100%** Client-Side

2. Application Overview

QuizForge operates as a single HTML file requiring no installation, server, or internet connection beyond the initial page load. All logic, question banks, user data, and interface rendering are handled entirely within the browser using vanilla JavaScript.

2.1 Core Purpose

The application serves Computer Science students and educators by providing:

- On-demand quiz generation from a curated 150-question bank
- Instant feedback with per-question explanations or end-of-quiz scoring
- Progress tracking tied to named student accounts
- Administrative visibility into class-wide performance
- Downloadable quiz files for offline or printed use

2.2 Technology Stack

Technology	Role	Details
HTML5	Document structure and semantic markup	Single-file architecture
CSS3	Styling, animations, responsive layout	CSS custom properties, Grid, Flexbox
JavaScript (ES6+)	Application logic, state management	Vanilla JS, no frameworks
Google Fonts	Typography (Space Mono + DM Sans)	Loaded via CDN link
Blob API	Quiz HTML file downloads	Client-side file generation

2.3 Deployment Model

The application is fully self-contained in a single .html file. Deployment requires only:

1. Copy the HTML file to any web-accessible location (or open locally)
2. No database, backend, or build step required
3. All student data persists in JavaScript memory for the session duration

Note: As a demonstration application, user data resets on page refresh. Production deployment would integrate localStorage, IndexedDB, or a backend API for persistence.

3. Authentication System

Version 2.0 introduces a complete role-based access control (RBAC) system with two distinct user roles: Student and Administrator. The login interface is the application entry point, replacing the direct quiz access of version 1.0.

3.1 Login Interface

The login screen provides a polished, role-aware entry experience:

- Role selection tabs (Student / Admin) with distinct colour theming — cyan for students, orange for administrators
- Contextual UI updates: labels, button colours, and input focus styles adapt to the selected role
- Inline demo credentials panel showing all valid test accounts
- One-click auto-fill for rapid demo access
- Error state handling with clear messaging for invalid credentials
- Enter key support for keyboard-first login

3.2 Demo Credentials

Username	Password	Role & Notes
alice	pass123	Student — 3 quiz attempts on record
bob	pass123	Student — 2 quiz attempts on record
carol	pass123	Student — 1 quiz attempt (top scorer)
dave	pass123	Student — no attempts yet
admin	admin123	Administrator — full dashboard access

3.3 Session Management

Authentication state is maintained in a JavaScript object for the session. On login:

- User object stored in currentUser variable with username, name, email, and role
- Header updates to display user badge with role indicator dot and name
- Sign-out button becomes visible
- Appropriate dashboard page is rendered based on role

On sign-out, all state is cleared and the user is returned to the login page with fields reset.

4. Student Experience

Students access a personalised dashboard immediately after login, combining performance analytics with the full quiz-taking interface in a single scrollable view.

4.1 Student Dashboard

The dashboard header displays three key performance metrics:

Metric	Description	Display
Quizzes Taken	Total number of completed quiz attempts	Colour-coded in cyan
Average Score	Mean percentage across all attempts	Colour-coded in green
Best Score	Highest single-quiz percentage achieved	Colour-coded in orange

4.2 Attempt History

Below the stats, a structured history table shows all past quiz attempts with:

- Topic chips indicating which subjects were covered (Java, OS, Environment)
- Question count for the attempt
- Colour-coded score pill: green (80%+), cyan (60-79%), red (below 60%)
- ISO date stamp of the attempt

New attempts are automatically prepended to the history on quiz completion, updating stats in real time.

4.3 Quiz Generation & Settings

Students configure and generate quizzes via three settings controls:

Setting	Options	Behaviour
Total Questions	10, 15, 25 (default), 30, or 50	Questions distributed proportionally across selected topics

Setting	Options	Behaviour
Answer Feedback	After each question, or submit all at end	Controls reveal timing and submit button visibility
Shuffle Options	Yes (randomise A/B/C/D) or No (keep original order)	Applied per-question at generation time

4.4 Topic Selection

Topic filter pills allow students to restrict quizzes to one or more subjects. At least one topic must remain selected at all times. Active topics are visually highlighted with their respective accent colours.

4.5 Quiz Interaction

Each question card displays:

- A colour-coded topic tag (Java/OS/Environment)
- The question text
- Four answer options in a 2x2 grid (1-column on mobile)
- Question counter (e.g. Q4/25)

On option selection in 'after each question' mode, the card immediately reveals the correct answer (green), marks the wrong choice if applicable (red), and displays a feedback banner. All options become unclickable. In 'submit at end' mode, the selected option is highlighted in cyan until submission.

4.6 Score & Progress

A progress bar at the top of the quiz section fills as questions are answered. On completion, a score panel displays the overall percentage, correct count, and per-topic breakdown. The student's attempt is simultaneously saved to their history.

5. Administrator Experience

Administrators access a dedicated management dashboard with three tabbed panels covering student oversight, attempt auditing, and account management.

5.1 Admin Overview Stats

Four summary cards at the top of the admin panel provide an at-a-glance platform snapshot:

- Total registered students
- Total quiz attempts across all users
- Platform-wide average score
- Total questions in the bank (150)

5.2 Students Tab

A card-based grid lists every student account showing their full name, username, email, quiz count, and colour-coded average score. A 'View' button opens a modal overlay with the student's full attempt history, including per-attempt topic breakdown, score, and date.

5.3 All Attempts Tab

A unified, date-sorted table shows every quiz attempt across all students, identifying the student by first name, the topics covered, question count, score (colour-coded), and date. This provides a complete audit trail of platform activity.

5.4 Add Student Tab

Administrators can register new student accounts via a four-field form (full name, username, email, initial password). On submission:

4. The new account is added to the in-memory user store
5. An empty history record is created for the student
6. All three tabs refresh to reflect the new entry
7. The form clears and a confirmation banner is displayed for 3 seconds

Existing students can be removed from the Manage tab with a confirmation prompt. Removal clears both the user account and all associated quiz history.

6. Question Bank

The question bank comprises 150 original multiple-choice questions — 50 per topic domain — written specifically to assess practical and conceptual knowledge at an undergraduate Computer Science level.

6.1 Topic Distribution

Domain	Count	Sub-topics Covered
Java Programming	50 questions	OOP, collections, concurrency, JVM internals, generics, streams, exception handling
Operating Systems	50 questions	Process management, scheduling, memory, file systems, IPC, deadlocks, synchronisation
Dev Environment	50 questions	Git workflows, Docker/containers, CI/CD, bash scripting, package management, cloud tools

6.2 Question Structure

Every question follows a consistent four-option MCQ format:

- Exactly four options labelled A, B, C, D
- Exactly one correct answer stored as a zero-based index
- Options cover common misconceptions and near-correct distractors
- Difficulty ranges from foundational recall to applied reasoning

6.3 Randomisation

At quiz generation time, the engine:

8. Samples questions proportionally from selected topics
9. Optionally shuffles the four options within each question (reassigning the correct answer index accordingly)
10. Shuffles the final question order

This three-layer randomisation ensures no two quizzes are identical even with the same settings.

6.4 Question Bank Browser

All 150 questions are browsable in the Question Bank section at the bottom of the student dashboard. Three tabs (Java, OS, Environment) display each question with its options, with the correct answer highlighted in green.

7. Quiz Download Feature

Any generated quiz can be exported as a standalone HTML file via the Download HTML button. The exported file is fully self-contained and functional offline, reproducing the complete quiz experience without requiring access to QuizForge.

7.1 Exported File Contents

- All quiz questions and answer options (in their randomised order)
- Answer keys embedded in JavaScript (not inspectable without viewing source)
- Full CSS styling matching the QuizForge dark theme
- Progress bar, score panel, and per-option feedback
- Submit all / reveal answers functionality

Exported files are named quiz_{n}q_{timestamp}.html and can be shared via email, LMS upload, or distributed on physical media.

8. UI/UX Design

8.1 Visual Design System

Element	Value	Usage
Background	#09090F (near-black)	Deep dark base with subtle blue tint
Surface	#111118	Raised containers
Card	#15151E	Question and dashboard cards
Java accent	#F97316 (orange)	Java topic, admin role, generate button
OS accent	#22D3EE (cyan)	OS topic, student role, selection state
Env accent	#A3E635 (lime)	Environment topic, correct answers, stats
Error	#F43F5E (rose)	Wrong answers, remove buttons, error states

8.2 Typography

Two typefaces create a technical yet readable hierarchy:

- Space Mono — monospace font used for all labels, tags, buttons, scores, and identifiers. Conveys a developer/terminal aesthetic.
- DM Sans — humanist sans-serif for body text, question content, and prose. Ensures readability at length.

8.3 Responsive Layout

The interface adapts across device widths:

- Full-width fluid layout up to 900px max content width
- Two-column option grid collapses to single column below 520px
- Config settings grid collapses from 3 columns to 1 below 580px
- Header tag line hides on narrow screens
- Add student form collapses from 2-column to 1-column grid

8.4 Animations

Page elements use staggered CSS keyframe animations (fup — fade up) on load. Interactive elements include hover lift effects on buttons, colour transitions on option selection, and a smooth progress bar fill transition.

9. Known Limitations & Future Enhancements

9.1 Current Limitations

- Session-only persistence: User accounts and quiz history reset on page refresh. Production use requires a backend or localStorage integration.
- In-memory credential storage: Passwords are stored in plain JavaScript objects. A real deployment must use server-side authentication with hashed passwords.
- Static question bank: New questions require editing the source file. A content management interface would improve maintainability.
- No password recovery: There is no mechanism to reset a forgotten password without admin intervention.
- Single-file limit: All 150 questions and all application logic are bundled in one file, increasing initial parse time on older devices.

9.2 Recommended Enhancements

11. Backend integration with REST API for persistent user accounts and quiz history
12. JWT or session-cookie based authentication replacing in-memory state
13. Admin question editor to add, edit, or retire bank questions without code changes
14. Student registration flow so students can self-enrol rather than relying on admin
15. Timed quiz mode with configurable countdown
16. Per-topic performance breakdown charts using Chart.js or similar
17. Export results as CSV for grade book integration
18. Password change and account settings page for students

10. Summary

QuizForge version 2.0 delivers a polished, fully functional computer science quiz platform within a single HTML file. The addition of role-based authentication transforms what was a standalone quiz generator into a multi-user educational tool capable of supporting a full class of students.

The Student role provides a personalised experience with historical tracking and instant performance feedback. The Administrator role provides the oversight tools — student management, attempt auditing, and account administration — that educators need to run assessments effectively.

With 150 professionally authored questions across Java, Operating Systems, and Development Environment domains, and a randomisation engine that ensures unique quizzes on every generation, QuizForge is ready for immediate academic use while providing a clear architectural foundation for a production deployment.